



Big Data System Design (MBAI 5110G)

Winter-2026

Week 5: Big Data Tools and Techniques

Lectured by:

Dilli P. Sharma, PhD

Outline

- In this lecture, we cover the following:
 - **Hadoop** Framework
 - Hadoop Components
 - Hadoop Distributed File System
 - Yet Another Resource Manager
 - MapReduce Algorithm
 - Hadoop Ecosystem
 - **Spark** Framework
 - Spark SQL
 - Spark Streaming
 - Spark Mlib
 - Spark GraphX
 - Spark's RDD and RDD operations
 - Using Spark's RDD APIs
 - Using Spark's SQL for Database Connectivity



Hadoop

- **What is Hadoop?**

- The Apache Hadoop¹ is a framework that allows for the distributed processing of large data sets (big data) across clusters of computers using simple programming models
- It is designed to scale up from single servers to thousands of machines, each offering local computation and storage
- Hadoop is a Big Data open-source software framework for reliable, scalable, and distributed computing



¹ <https://hadoop.apache.org/>

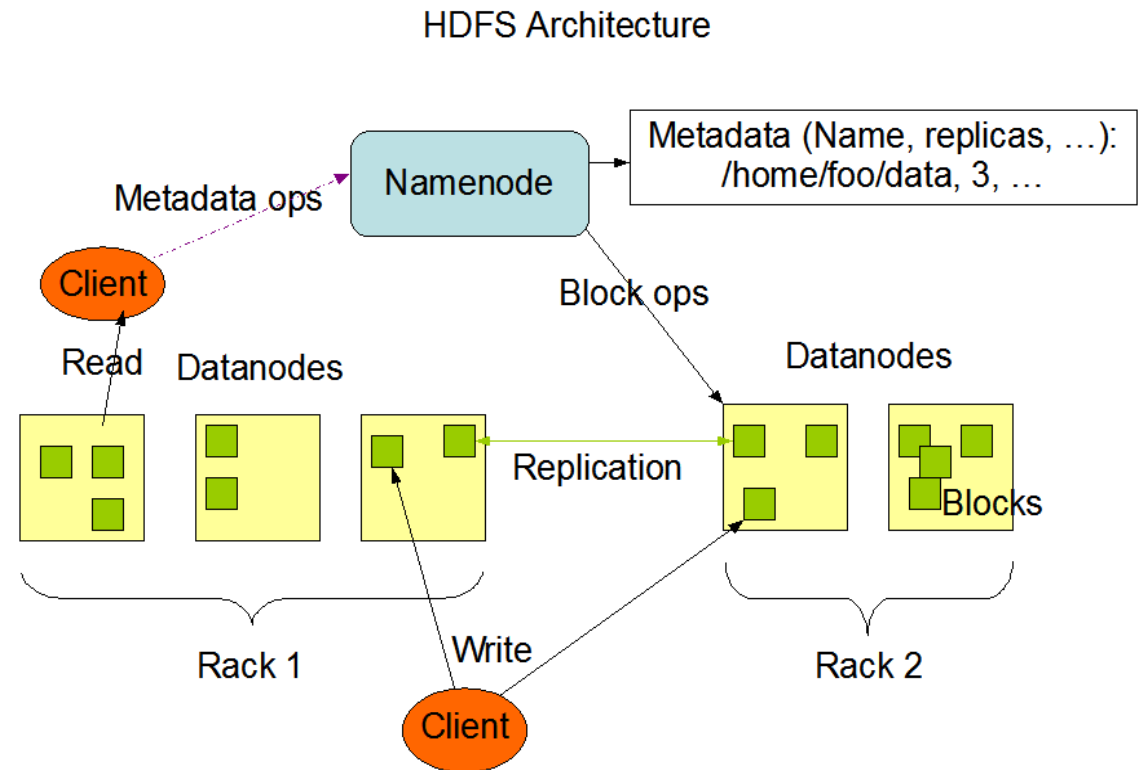
Hadoop(cont.)

- **Main modules of Hadoop are:**
 - **Hadoop Common:** The common utilities that support the other Hadoop modules.
 - **Hadoop Distributed File System (HDFS™):**
A distributed file system that provides high-throughput access to application data.
 - **Hadoop YARN:** A framework for job scheduling and cluster resource management.
 - **Hadoop MapReduce:** A YARN-based system for parallel processing of large data sets.



HDFS

- **HDFS** is a core (backbone) component of the Hadoop ecosystem
- **HDFS** makes possible to store different types of large data sets and handles distributed processing
 - Structured, unstructured and semi structured data
- It has two components: name node (master) and data node (slave)



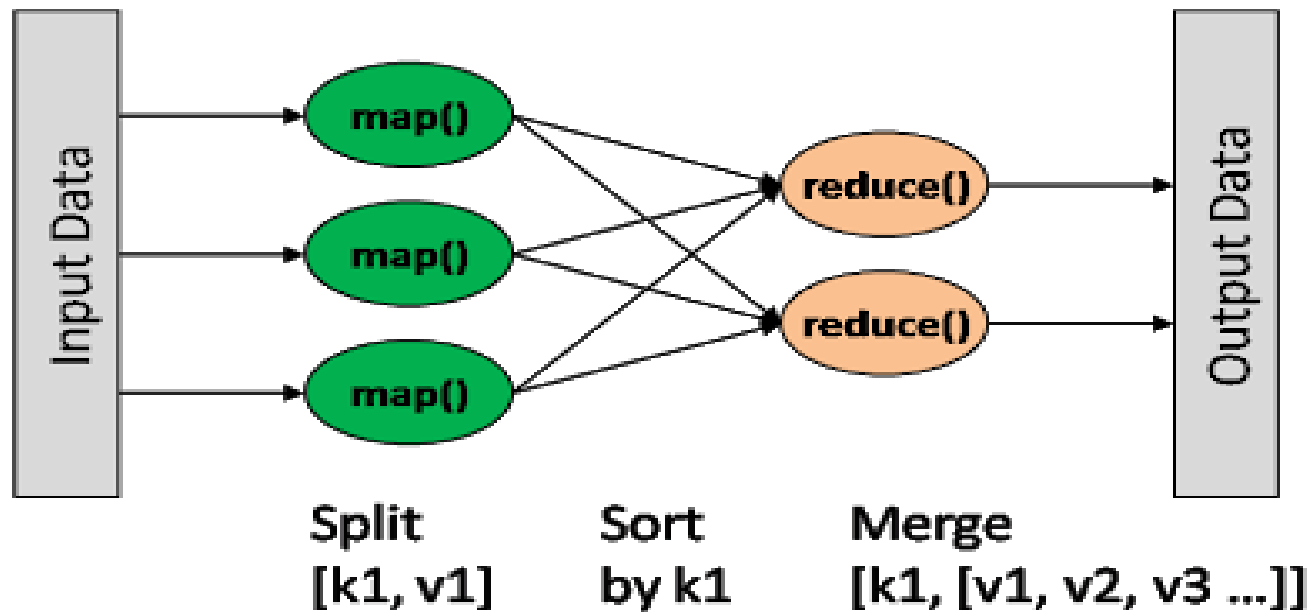
Yet Another Resource Navigator (YARN)

- **YARN** is considered as the brain Hadoop Ecosystem.
- It performs all your processing activities by allocating resources and scheduling tasks.
- **YARN has two components :**
 - **ResourceManager (Master):**
 - This is the master daemon.
 - It manages the assignment of resources such as CPU, memory, and network bandwidth.
 - **NodeManager (Slave):**
 - This is the slave daemon
 - It reports the resource usage to the Resource Manager
 - It handles requests at every DataNode



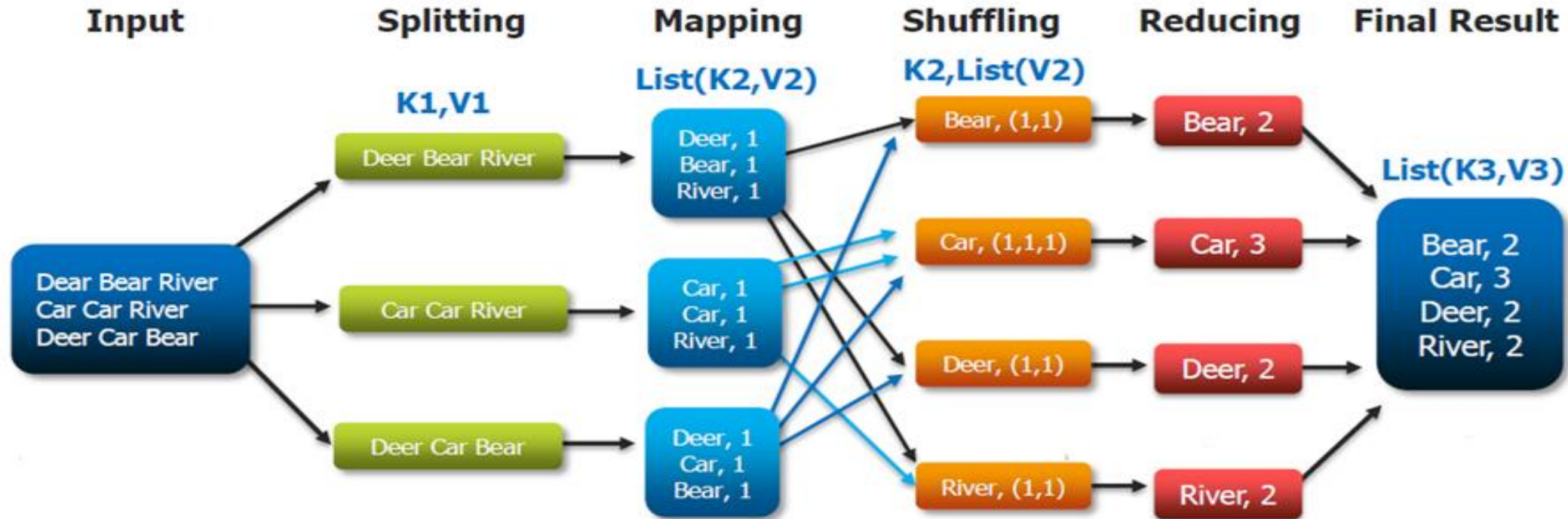
MapReduce

- **MapReduce** is a programming model that allows us to perform parallel and distributed processing on huge data sets.
- It has two phases/functions:
 - Map: Map function performs actions like filtering , grouping and sorting
 - Reduce: Reduce function aggregates and summarizes the results produced by map function



How MapReduce Works?

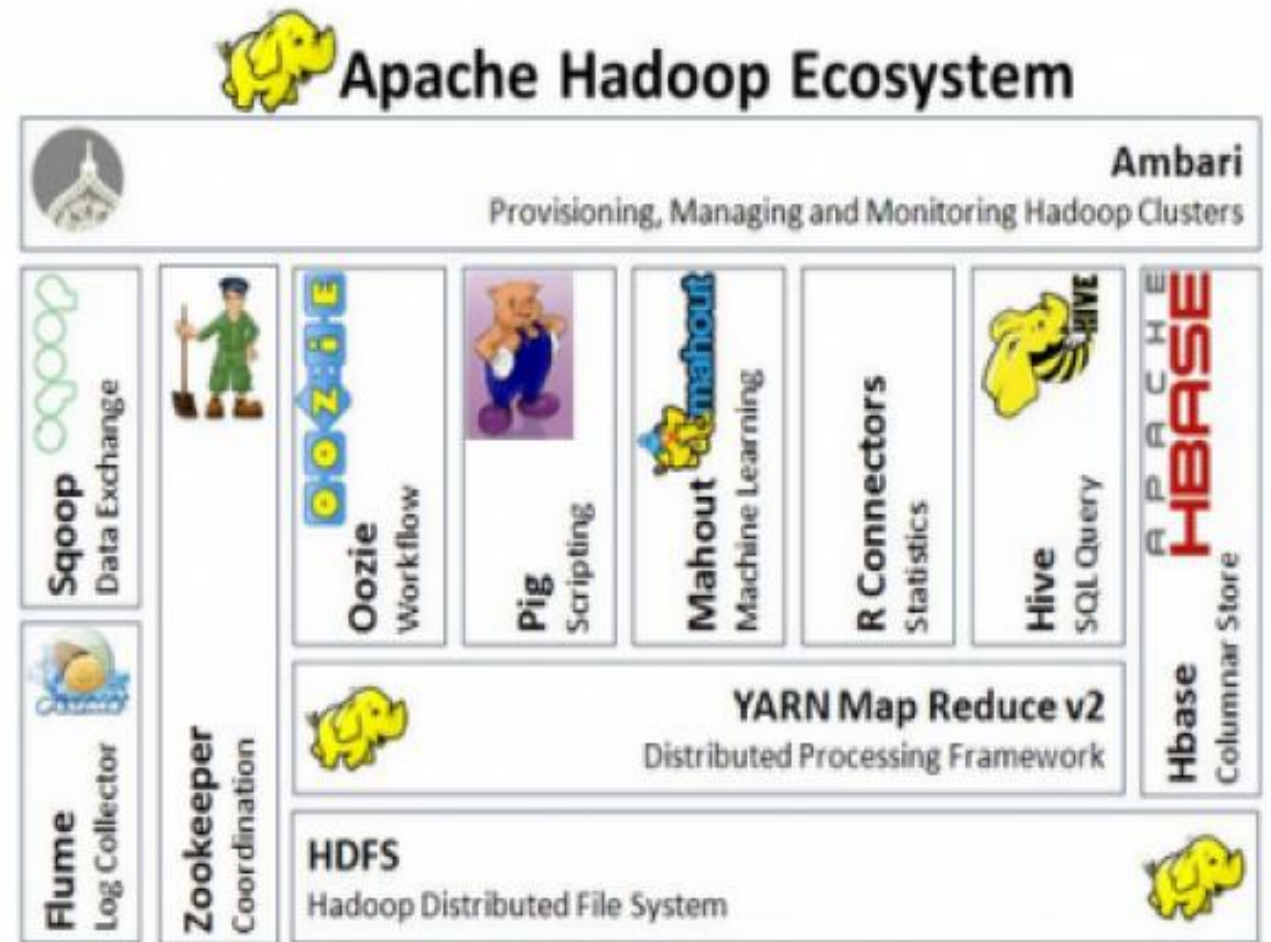
The Overall MapReduce Word Count Process



Hadoop Ecosystem

- **Components of Hadoop ecosystem:**

- **HDFS:** Hadoop Distributed File System
- **YARN:** Yet Another Resource Negotiator
- **MapReduce:** Data processing using programming
- **Spark :** In-memory Data Processing
- **PIG, HIVE:** Data Processing Services using Query (SQL-like)
- **HBase :** NoSQL Database
- **Mahout, Spark MLlib:** Machine Learning
- **Apache Drill:** SQL on Hadoop
- **Zookeeper:** Managing Cluster
- **Oozie:** Job Scheduling
- **Flume, Sqoop:** Data Ingestion Services
- **Solr & Lucene:** Searching & Indexing
- **Ambari:** Provision, Monitor and Maintain cluster



Spark Framework



Introduction to Apache Spark

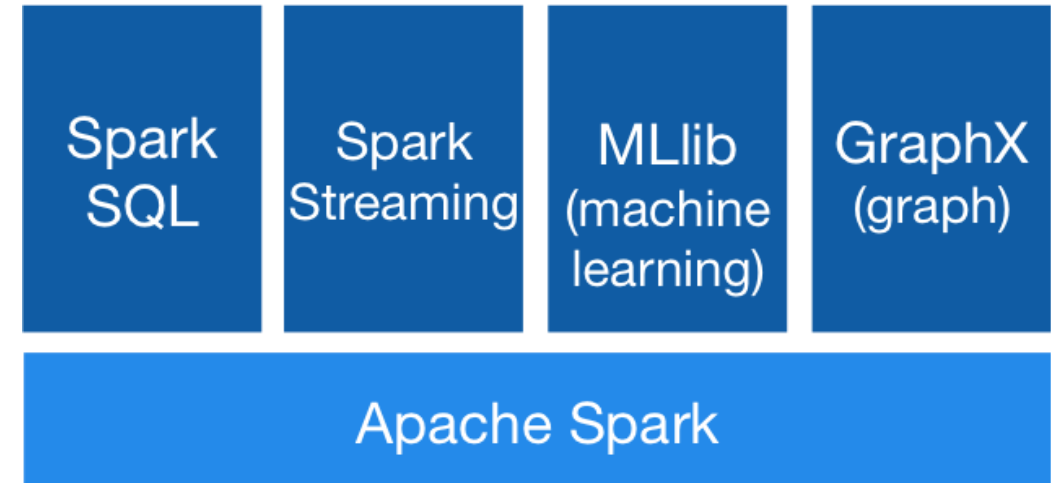
- **Why Spark?**

- **Hadoop MapReduce** is a programming model for processing big data sets with a parallel, distributed algorithm.
- However, MapReduce is the sequential multi-step process it takes to run a job:
 - Reads data from the cluster, performs operations, and writes the results back to HDFS.
 - Each step requires a disk read and write; MapReduce jobs are slower due to the latency of **disk I/O**.
- **Spark** was created to address the limitations of MapReduce by doing processing **in-memory**, reducing the number of steps in a job, and reusing data across multiple parallel operations.
 - Spark runs multiple **times faster** than MapReduce, especially when doing machine learning and interactive analytics.



What is Spark?

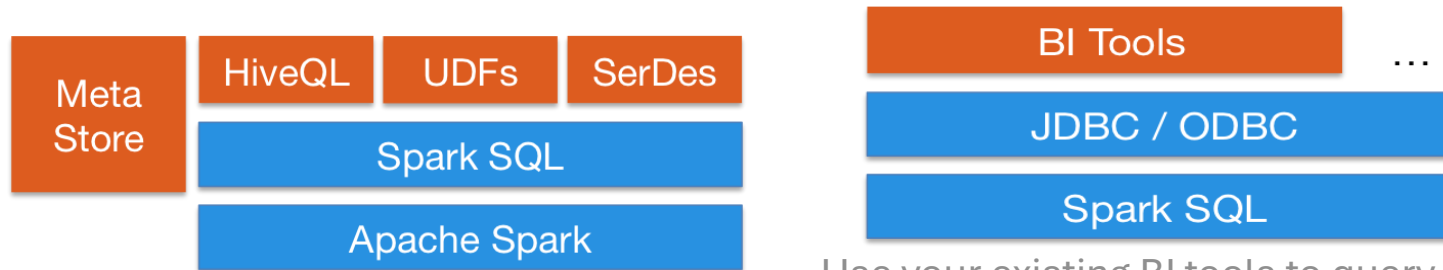
- **Apache Spark¹** is a unified analytics engine for large-scale data processing.
 - **Speed:** Spark achieves high performance for both batch and streaming data in state-of-the-arts
 - **Ease of Use:** Write applications quickly in Java, Scala, Python, R, and SQL.
 - **Generality:** Combine SQL, streaming, and complex analytics ML applications
 - **Runs Everywhere:** Spark runs on Hadoop, Apache Mesos, Kubernetes, standalone, or in the cloud. It can access diverse data sources:
 - HDFS, Alluxio, Apache Cassandra, Apache HBase, Apache Hive, etc.



¹<https://spark.apache.org/>

Spark SQL

- **Spark SQL** is Apache Spark's module for working with structured data.
 - **Integrated:** Seamlessly mix SQL queries with Spark programs.
 - **Uniform Data Access:** Connect to any data source the same way
 - **Hive Integration:** Run SQL or HiveQL queries on existing warehouses.
 - **Standard Connectivity:** Connect through JDBC or ODBC.



Use your existing BI tools to query big data.

Spark SQL can use existing Hive metastores, SerDes, and UDFs.

Apply functions to results of SQL queries:

```
results = spark.sql(  
    "SELECT * FROM people"  
)  
names = results.map(lambda p: p.name)
```

Query and join different data sources:

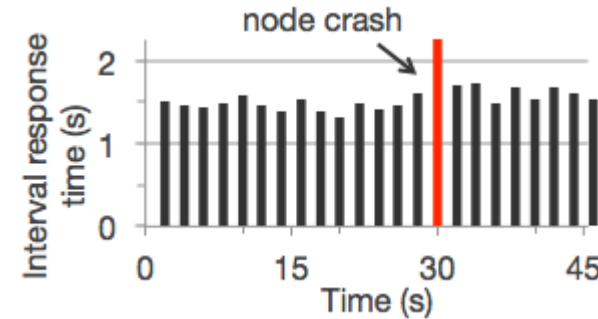
```
spark.read.json("s3n://...")  
    .registerTempTable("json")  
results = spark.sql(  
    """SELECT *  
    FROM people  
    JOIN json ...""")
```

Spark Streaming

- **Spark Streaming** makes it easy to build scalable fault-tolerant streaming applications.
 - **Ease of Use:** Build applications through high-level operators.
 - **Fault Tolerance:** Stateful exactly-once semantics out of the box.
 - **Spark Integration:** Combine streaming with batch and interactive queries.
 - **Deployment Options:** Spark Streaming can read data from HDFS, Flume, Kafka, Twitter and ZeroMQ.

```
TwitterUtils.createStream(...)
    .filter(_.getText.contains("Spark"))
    .countByWindow(Seconds(5))
```

Counting tweets on a sliding window



```
stream.join(historicCounts).filter {
  case (word, (curCount, oldCount)) =>
    curCount > oldCount
}
```

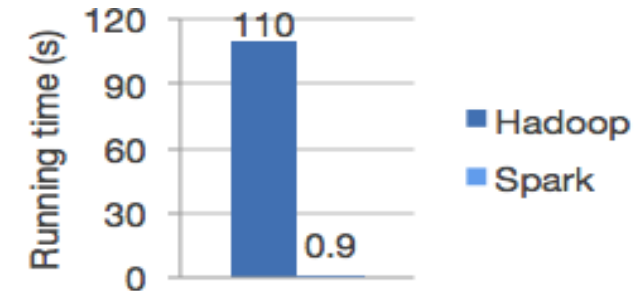
Find words with higher frequency
than historic data

Spark MLlib

- **MLlib** is Apache Spark's scalable machine learning library
 - **Ease of Use:** Usable in Java, Scala, Python, and R.
 - **Performance:** High-quality algorithms, 100x faster than MapReduce.
 - **Runs Everywhere:** Spark runs on Hadoop, Apache Mesos, Kubernetes, standalone, or in the cloud, against diverse data sources.
 - **Algorithms:** MLlib contains many ML algorithms and utilities.
 - **ML Algorithms:**
 - Classification, Regression, Clustering, etc.
 - **ML Workflow utilities:**
 - Feature transformation (standardization, normalization, etc.)
 - ML pipelines, Model evaluations, etc.
 - **Other utilities:**
 - Distributed linear algebra: SVD, PCA, etc.
 - Statistics: summary statistics, hypothesis testing, etc.

```
data = spark.read.format("libsvm")\
    .load("hdfs://...")
model = KMeans(k=10).fit(data)
```

Calling MLlib in Python



Logistic regression in Hadoop and Spark

Spark GraphiX

- GraphX is Apache Spark's API for graphs and graph-parallel computation
 - **Flexibility:** Seamlessly work with both graphs and collections.
 - GraphX unifies ETL, exploratory analysis, and iterative graph computation within a single system.
 - **Speed:** Comparable performance to the fastest specialized graph processing systems.
 - **Algorithms:** Choose from a growing library of graph algorithms.
 - GraphX comes with a variety of graph algorithms:
 - PageRank
 - Connected components
 - Label propagation
 - SVD++
 - Strongly connected components
 - Triangle count

```
graph = Graph(vertices, edges)
messages = spark.textFile("hdfs://...")
graph2 = graph.joinVertices(messages)
{
  (id, vertex, msg) => ...
}
```

Using GraphX in Scala

Spark's Resilient Distributed Datasets (RDD)

- **Spark** application consists of a driver/main program that runs the user's main function and executes various parallel operations on a cluster
- **RDD** is an immutable, partitioned collection of elements that can be operated on in parallel
- There are two ways to create RDDs:
 - *Parallelizing* collection:
 - **Parallelized collections** are created by calling SparkContext's `parallelize` method
 - *Referencing a dataset* in an external storage system (HDFS, HBase, or any data source offering a Hadoop Input Format)
 - **External Datasets** RDDs can be created using SparkContext's `textFile` method.

RDD Operations

- **RDDs** support two types of operations:
 - **Transformations**
 - *Transformations* create a new dataset from an existing one
 - Examples: `map()`, `filter()`, `flatMap()`, `mapPartitions()`, `sample()`, `union()`, `intersection()`, `distinct()`, `reduceByKey()`, `groupByKey()`, `aggregateByKey()`, `sortByKey()`, `join()`, *etc.*
 - **Actions:**
 - *Actions* return a value to the driver program after running a computation on the dataset
 - Examples: `reduce()`, `count()`, `collect()`, `first()`, `take()`, `saveAsTextFile()`, `countByKey()`, `foreach()`, *etc.*

Spark's Shuffle Operations

- Certain operations within Spark trigger an event known as the **shuffle**.
- The **shuffle** is Spark's mechanism for **re-distributing data** so that it's grouped differently across partitions
- Spark shuffling triggers for transformation operations like *groupByKey(), reduceByKey(), join(), union(), groupBy()* e.t.c
- Spark Shuffle is an expensive operation since it involves the following
 - Disk I/O
 - Involves data serialization and deserialization
 - Network I/O
- Shuffle behavior can be tuned by adjusting a variety of configuration parameters

Using Spark's RDD API Examples in Python, Scala and Java

Word Count: in this example, we use a few transformations to build a dataset of (String, Int) pairs called counts and then save it to a file.

Python

```
text_file = sc.textFile("hdfs://...")
counts = text_file.flatMap(lambda line: line.split(" ")) \
    .map(lambda word: (word, 1)) \
    .reduceByKey(lambda a, b: a + b)
counts.saveAsTextFile("hdfs://...")
```

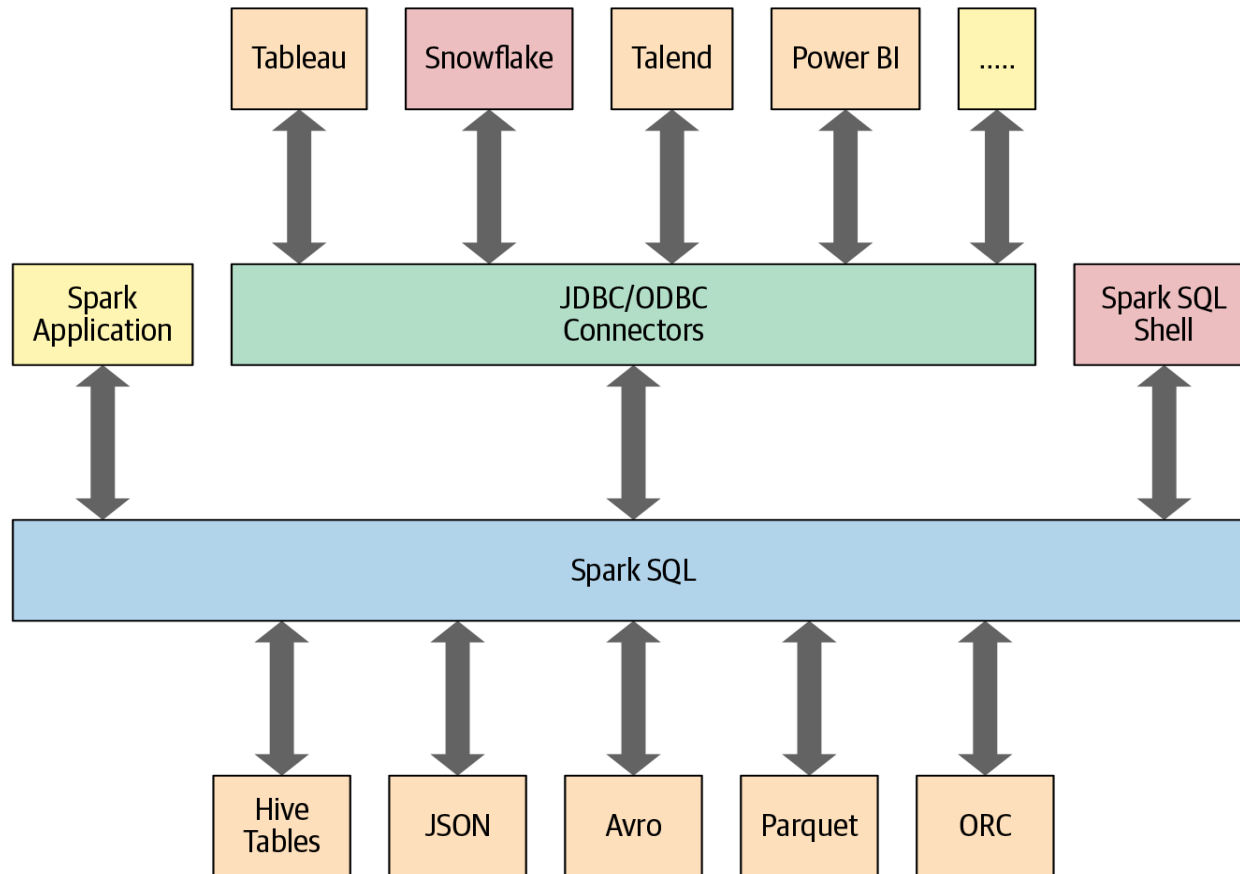
Scala

```
val textFile = sc.textFile("hdfs://...") val counts =
textFile.flatMap(line => line.split(" "))
    .map(word => (word, 1))
    .reduceByKey(_ + _)
counts.saveAsTextFile("hdfs://...")
```

Java

```
JavaRDD<String> textFile = sc.textFile("hdfs://...");
JavaPairRDD<String, Integer> counts = textFile
    .flatMap(s -> Arrays.asList(s.split(" ")))
    .iterator()) .mapToPair(word -> new Tuple2<>(word, 1))
    .reduceByKey((a, b) -> a + b);
counts.saveAsTextFile("hdfs://...");
```

Using Spark for Database Connectivity



```
# Creates a DataFrame based on a table named "people"
# stored in a MySQL database.
url =
"jdbc:mysql://yourIP:yourPort/test?user=yourUsername;pass
word=yourPassword"
df = sqlContext \
    .read \
    .format("jdbc") \
    .option("url", url) \
    .option("dbtable", "people") \
    .load()

# Looks the schema of this DataFrame.
df.printSchema()

# Counts people by age
countsByAge = df.groupBy("age").count()
countsByAge.show()

# Saves countsByAge to S3 in the JSON format.
countsByAge.write.format("json").save("s3a://...")
```

Reading Data from CSV file

```
from pyspark.sql import SparkSession
# Create a SparkSession
spark = (SparkSession
        .builder
        .appName("SparkSQLExampleApp")
        .getOrCreate())
# Path to data set
csv_file = "/databricks-datasets/learning-spark-v2/flights/departuredelays.csv"
# Read and create a temporary view # Infer schema (note that for larger files you
# may want to specify the schema)
df = (spark.read.format("csv")
      .option("inferSchema", "true")
      .option("header", "true")
      .load(csv_file))
df.createOrReplaceTempView("us_delay_flights_tbl")
```

Textbook/References

- Chapter -5: Driving Big Data with Hadoop Tools and Technologies from the TextBook: Big Data: Concepts, Technology, and Architecture, Balamurugan Balusamy, Nandhini Abirami R, Seifedine Kadry, Amir H. Gandomi, Wiley, 2021
- Hadoop: <https://hadoop.apache.org/>
- Spark: <https://spark.apache.org/>
- Online References and Tutorials:
 - [PySpark Overview](#)
 - [QuickStart: DataFrame](#)
 - [Spark SQL](#)
 - [Structured Streaming Programming](#)
 - [Resilient Distributed Datasets \(RDDs\) Programming](#)
 - [Spark Tutorials](#)

Next Session: Big Data Pipeline

Q/A

Thank you!